# SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING PRACTICAL ASSIGNMENT

## 1. Student Details

| | |
|---|---|
| **Student Name** | Pratik Akrur Pandit |
| **PRN** | 202501110116 |
| **Division** | B |
| **Batch** | B2 |
| **Roll No.** | 202501110116 |
| **Date of Submission** | 26/10/2026 |

## 2. Practical Details

| | |
|---|---|
| **Practical No.** | 9 |
| **Practical Title** | std::move in utility |
| **Date Performed** | 24/10/2026 |
| **Faculty Name** | Shruti Gadewar |

## 3. Problem Statement

Write a C++ program that demonstrates the use of **std::move()** from the <utility> header. The program must show how std::move() converts an lvalue into an rvalue reference so that the resources of one object are transferred (moved) to another object instead of being copied, and must compare the copy constructor with the move constructor.

## 4. Objectives

1. To understand lvalue references (&) and rvalue references (&&) in C++.
2. To use std::move() to transfer resources from one object to another efficiently.
3. To implement a move constructor and compare it with a copy constructor.

## 5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain the purpose of the move constructor introduced in C++11.
2. Use std::move() to move strings, vectors and user-defined objects without unnecessary copying.
3. Distinguish between the copy constructor and the move constructor and state the condition of an object after it has been moved from.

## 6. Theory / Concept

The **move constructor** was introduced in C++11. Its purpose is to steal or move as many resources as it can from the source (original) object, as fast as possible, because the source does not need to have a meaningful value anymore and/or is going to be destroyed in a moment anyway. This avoids unnecessarily creating copies of an object and makes efficient use of resources.

The **copy constructor** uses lvalue references, marked with one ampersand (&), while the **move constructor** uses rvalue references, marked with two ampersands (&&).

**std::move()** is a function used to convert an lvalue reference into an rvalue reference. It is used to move the resources from a source object, i.e. for efficient transfer of resources from one object to another. It is defined in the <utility> header. Note that std::move() itself does not move anything; it only casts the object so that the move constructor (or move assignment) is selected.

**Syntax:**

```
template< class T >
typename std::remove_reference<T>::type&& move(T&& t) noexcept;      // since C++11, until C++14

template< class T >
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept;        // since C++14
```

After an object has been moved from, it remains valid but its value is unspecified (for example, a moved-from string or vector is usually empty), so it should only be destroyed or assigned a new value.

## 7. Algorithm / Pseudocode

```
START
Step 1: Include <iostream>, <string>, <vector> and <utility>.
Step 2: Create string str1 = "Hello, C++ Programming" and display it.
Step 3: Move str1 into str2 using str2 = std::move(str1);
Step 4: Display str1 and str2 after the move (str1 is now empty).
Step 5: Create vector v1, move it into v2 using std::move(), and display the
        sizes of both vectors.
Step 6: Define class Buffer with a copy constructor (const Buffer&) and a
        move constructor (Buffer&&).
Step 7: Create b1, copy it into b2, move it into b3 using std::move(), and
display the size of each object. STOP
```

## 08. Technology / Tools / Libraries Used

| Technology / Tool | Purpose |
|---|---|
| Programming Language | C++ (C++11 standard) |
| IDE / Platform | Terminal / text editor on Linux |
| Compiler | g++ (GCC) |
| Libraries / Packages | <iostream> (input/output), <string>, <vector>, <utility> (std::move) |

## 09. Implementation / GitHub Repository

| | |
|---|---|
| **GitHub Repository URL** | https://github.com/panditpratik/OOP- |
| **Repository Name** | **Problem-Solving-Using-OOP-in-Cpp** |
| **Branch Used** | main |
| **Commit ID / Latest Commit** | 7f8fc4031843bc9245fef14b740b25841f3d265b |
| **Date of Last Commit** | 26/10/26 |

## 10. Test Cases / Input and Output

| Test Case | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| TC1 string before move | str1 = "Hello, C++ Programming" | str1 = Hello, C++ Programming | str1 = Hello, C++ Programming | Pass |

| | | | | |
|---|---|---|---|---|
| TC2 str1 after move | str2 = std::move(str1) | str1 = (empty) | str1 = (empty) | Pass |
| TC3 str2 after move | str2 = std::move(str1) | str2 = Hello, C++ Programming | str2 = Hello, C++ Programming | Pass |
| TC4 vector before move | v1 = {1, 2, 3, 4, 5} | v1.size() = 5 | v1.size() = 5 | Pass |
| TC5 vector after move | v2 = std::move(v1) | v1.size() = 0, v2.size() = 5 | v1.size() = 0, v2.size() = 5 | Pass |
| TC6 Buffer b2 = b1 | lvalue | Copy constructor: copied 5 ints | Copy constructor: copied 5 ints | Pass |
| TC7 Buffer b3 = std::move(b1) | rvalue | Move constructor: moved 5 ints (no copy) | Move constructor: moved 5 ints (no copy) | Pass |
| TC8 Buffer sizes after move | b1, b2, b3 | b1.getSize() = 0, b2.getSize() = 5, b3.getSize() = 5 | b1.getSize() = 0, b2.getSize() = 5, b3.getSize() = 5 | Pass |

## 11. Results / Observations

- After str2 = std::move(str1), str2 held the text "Hello, C++ Programming" and str1 became empty, showing that the string's resources were transferred instead of copied.
- After the vector was moved, v2 contained all five elements and v1.size() became 0.
- For the Buffer class, b2 = b1 (an lvalue) invoked the copy constructor, which allocated new memory and copied every element, whereas std::move(b1) (an rvalue) invoked the move constructor, which simply took over b1's memory.
- After the move, b1.getSize() was 0, which shows that a moved-from object is left in a valid but empty state.

**Screenshots / Graphs / Output:**

Pratical_assignment_9.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <utility>
using namespace std;

// Class with copy constructor and move constructor
class Buffer {
    int *data;
    int size;

public:
    Buffer(int n) : data(new int[n]), size(n) {
        for (int i = 0; i < size; i++)
            data[i] = i + 1;

        cout << "Constructor: allocated " << size
             << " ints" << endl;
    }

    // Copy constructor
    Buffer(const Buffer &other)
        : data(new int[other.size]), size(other.size) {
        for (int i = 0; i < size; i++)
            data[i] = other.data[i];

        cout << "Copy constructor: copied " << size
             << " ints" << endl;
    }

    // Move constructor
    Buffer(Buffer &&other) noexcept
        : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;

        cout << "Move constructor: moved " << size
             << " ints (no copy)" << endl;
    }

    ~Buffer() {
        delete[] data;
    }

    int getSize() const {
        return size;
    }
```

```cpp

int main() {
    cout << "===== 1. std::move with string =====" << endl;

    string str1 = "Hello, C++ Programming";

    cout << "Before moving:" << endl;
    cout << "str1 = " << str1 << endl;

    string str2 = std::move(str1);

    cout << "\nAfter moving:" << endl;
    cout << "str1 = " << str1 << endl;
    cout << "str2 = " << str2 << endl;

    cout << endl << "===== 2. std::move with vector =====" << endl;

    vector<int> v1 = {1, 2, 3, 4, 5};

    cout << "Before moving: v1.size() = "
         << v1.size() << endl;

    vector<int> v2 = std::move(v1);

    cout << "After moving: v1.size() = "
         << v1.size()
         << ", v2.size() = " << v2.size() << endl;

    cout << endl
         << "===== 3. Copy constructor vs Move constructor ====="
         << endl;

    Buffer b1(5);

    Buffer b2 = b1;             // Copy constructor
    Buffer b3 = std::move(b1);  // Move constructor

    cout << "\nBuffer sizes after operations:" << endl;
    cout << "b1 size = " << b1.getSize() << endl;
    cout << "b2 size = " << b2.getSize() << endl;
    cout << "b3 size = " << b3.getSize() << endl;

    cout << "\nProgram ended successfully." << endl;

    return 0;
}
```

- **Output:-**

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== 1. std::move with string =====
Before moving:
str1 = Hello, C++ Programming

After moving:
str1 =
str2 = Hello, C++ Programming

===== 2. std::move with vector =====
Before moving: v1.size() = 5
After moving: v1.size() = 0, v2.size() = 5

===== 3. Copy constructor vs Move constructor =====
Constructor: allocated 5 ints
Copy constructor: copied 5 ints
Move constructor: moved 5 ints (no copy)

Buffer sizes after operations:
b1 size = 0
b2 size = 5
b3 size = 5

Program ended successfully.
PS D:\All Programs\OOP(C++)>
```

## 12. Conclusion

In this practical, a C++ program was implemented to demonstrate std::move() from the <utility> header. Strings, vectors and a user-defined Buffer class were moved instead of copied, and the copy constructor was compared with the move constructor. I learned that std::move() converts an lvalue into an rvalue reference so that resources can be transferred efficiently, which avoids unnecessary copies and improves performance.

## 13. Assignment Questions

**Q1. What is std::move()? In which header is it defined?**

Answer: std::move() is a function, introduced in C++11, that converts an lvalue reference into an rvalue reference. It is used to move the resources from a source object for efficient transfer from one object to another. It is defined in the <utility> header. It does not move anything by itself; it only allows the move constructor or move assignment operator to be selected.

**Q2. What is the state of an object after it has been moved from?**

Answer: After std::move(), the source object is left in a valid but unspecified state. For standard containers such as string and vector it is usually empty (as seen in the output, where str1 printed nothing and v1.size() was 0). The moved-from object should not be relied upon for its old value; it can safely be destroyed or assigned a new value.

## 14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

| Student Name | PRN | Signature |
|---|---|---|
| Pratik Akrur Pandit | 202501110116 |  |